

Outil de gestion d'images



Le projet devra être réalisé en **Binôme** (ou Trinôme après accord). Les personnes seules qui n'arrivent pas à trouver de binôme pourront s'en voir affecter un d'office par l'enseignant. Les sources seront à remettre sous forme d'une archive zip à travers la plateforme communities. **Aucun rendu par mail ne sera accepté.**

Présentation générale du projet

L'objectif du projet est de réaliser un outil de gestion de bibliothèque d'images simplifié avec des fonctions de sécurité. Pour cela, plusieurs fonctionnalités sont attendues dont des fonctions de bases :

- Mise en place d'opérations de traitement d'images.
- Gestion de tags pour identifier et chercher les images dans la bibliothèque.
- Mise en place d'une fonction de "chiffrement" d'image.

puis des fonctions un peu plus avancées pour les plus rapides d'entre vous :

- Mise en place d'un système de persistance des données (filtres + tags associés à une image).



Pensez à réaliser la conception générale de votre projet avant de vous lancer dans le développement. **La qualité de la conception logicielle interviendra de manière importante dans la note.**

A minima, devrez avoir une application respectant le **paradigme MVC** à la fin du projet. Cela veut dire qu'il faut que votre interface soit décrite en FXML et que vous utilisiez des contrôleurs pour gérer les interactions utilisateurs.



Attention, vos contrôleurs JavaFX ne doivent pas contenir toute la logique de votre application. **Pensez à faire plusieurs classes !**

Application réduite

Dans cette première partie commencez par:

- Essayer de lire une image et de l'afficher dans une fenêtre JavaFX. Pour cela, appuyez vous sur l'API de la classe **Image** et de la classe **ImageView**.
- Pour rendre votre projet plus lisible, créez s'il n'existe pas déjà, un nouveau répertoire nommé "resources" qui contiendra toutes les images à manipuler.
- Ajoutez ensuite un **FileChooser** vous permettant de sélectionner l'image à afficher.

Les transformations d'image

Dans cette partie, vous proposerez différentes transformations à appliquer aux images. Pour cela, on suppose que les images dont on dispose peuvent être représentées via une matrice de pixels de dimensions m par n . Chaque pixel a une couleur caractérisée par trois composantes RGB (comprises entre 0 et 255) qui représentent chacune une couleur : rouge, vert et bleu. Appuyez-vous une fois de plus sur la documentation du paquetage `javafx.scene.image` pour accéder aux pixels d'une image et comprendre comment manipuler les composantes RGB.

La première chose à mettre en place est :

- un système de transformation permettant d'effectuer une rotation ou une symétrie d'image.

Dans un deuxième temps, vous proposerez un système de filtres pour traiter une image. Pour cela quelques contraintes de conception vous sont imposées. Vous devrez à minima :

- Utiliser la notion d'héritage et les interfaces pour éviter la redondance de code.
- Proposer des filtres permettant:
 - d'échanger les composantes (R, G, B) pour obtenir (G, B, R)
 - d'afficher une image en noir et blanc (moyenne des composantes (R, G, B))
 - d'afficher une image en sepia (voir : <http://stackoverflow.com/questions/1061093/how-is-a-sepia-tone-created>)
 - de réaliser le filtre de Prewitt pour effectuer de la détection de contours sur votre image.
- Intégrez ces filtres au sein de votre interface pour permettre à l'utilisateur de les appliquer sur une image de son choix.

Support des tags et sauvegarde des transformations



Cet exercice, bien que moins flashy que le précédent est très important dans la notation globale du projet.

Dans cet exercice, nous allons mettre au point un système permettant d'effectuer des recherches intuitives sur les images. En effet, même si les arborescences classiques restent le moyen privilégié pour stocker des informations sur un disque dur, elles ne sont souvent pas les plus adaptées pour retrouver rapidement de l'information. Pour cela nous allons proposer un système de Meta-données que nous stockerons dans des fichiers textes (version de base) ou des fichiers JSON (version avancée).

Pour simplifier, toutes les métadonnées de la bibliothèque pourront être stockées dans le même fichier. La structuration du fichier est laissée à votre libre appréciation mais devra quand même s'appuyer sur les informations suivantes :

- Pour les tags, le fichier de métadonnée associé à une image contiendra une liste de tous les tags que l'utilisateur aura défini pour l'image.

- Pour les transformations, une liste des différentes transformations à appliquer (avec leurs paramètres) sera stockée. Cette liste permettra de réappliquer les transformations dans le même ordre au chargement de l'image.

La version texte est la plus rapide à mettre en oeuvre mais n'est pas la plus pratique à manipuler (en particulier pour le chargement et la création des objets). En général, on privilégie le format JSON pour son expressivité. Son utilisation en Java est relativement simple et s'appuie sur JSON-P ou JSON-B. Pour résumer de manière très rapide, ces bibliothèques permettent de serialiser un objet directement dans le format de fichier attendu (JSON).

Une autre bibliothèque encore plus simple est **Jackson**. Vous pouvez en trouver une présentation rapide sur le lien suivant : <https://www.baeldung.com/jackson-object-mapper-tutorial>. **Seule cette librairie est autorisée dans le cadre de ce projet !**



Dans une version avancée (bonus), vous pouvez regarder le patron de conception DAO (Data Access Object) qui peut vous simplifier grandement la quantité de développement à réaliser pour passer des fichiers textes aux fichiers JSON.

Un peu de sécurité

Pour que les étudiants intéressés par le master Cryptis ne se sentent pas floués par le début du projet, une fonction de sécurité supplémentaire sera ajoutée pour permettre de conserver des images de manière sécurisée. Le concept est assez simple : l'idée est de mélanger les pixels d'une image dans un ordre prédictif qui dépendra d'un mot de passe fourni par l'utilisateur.

Pour cela, nous nous appuierons sur l'objet `SecureRandom` fourni par la JDK. Comme dans de très nombreux langages, ce générateur de nombres aléatoires est contrôlé par une graine (seed) qui permet de générer des suites prédictibles de résultats. Ainsi, si vous utilisez deux fois de suite la même seed pour deux instances de générateurs de nombres aléatoires, ceux-ci vont effectuer les mêmes tirages.

La graine est en général un nombre entier. Pour passer d'un mot de passe fourni par l'utilisateur à un nombre entier servant à initialiser notre générateur de nombres aléatoires, une solution classique consiste à passer par une fonction de hachage. Ces fonctions standards en sécurité se retrouvent dans la JDK dans le paquetage `java.security`. Parmi toutes les fonctions proposées, les plus classiques sont MD5 (considérée comme non sûre donc à éviter), SHA-1 (idem) et SHA-256.

La classe `SecureRandom` (voir : <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/security/SecureRandom.html>) permet de simplifier leur utilisation en spécifiant le type de fonction de hachage à utiliser pour générer les nombres aléatoires et ce en acceptant des graines fournies sous forme de tableau d'octets.

D'un point de vue ergonomie ce filtre spécial modifiera directement l'image d'origine lorsqu'il sera appliqué et la seule méta donnée associée à conserver est le nom du filtre. Bien entendu, le mot de passe pour déchiffrer l'image ne doit pas être conservé.

Les fonctions avancées

Les exercices précédents vous donnent les fonctionnalités indispensables pour le projet mais vous pouvez aller plus loin (et donc avoir une meilleure note :smiley:).

Une première amélioration consiste à mettre en place un système de persistance des méta données dans une base de donnée plutôt que dans un fichier texte ou JSON. Si vous avez bien conçu votre application cette modification ne devrait pas entraîner de changements importants dans votre code. **Utilisez une base H2 pour réaliser vos tests.**



H2 est une base de données relationnelle de petite taille (~1Mo) qui utilise le langage SQL. Elle est très flexible pour du prototypage ou des tests au même titre qu'Apache Derby ou HSQLDB par exemple. Elle s'interface avec les programmes Java à travers JDBC.

La base de donnée H2 s'installe très simplement en dézipant une archive que vous pouvez retrouver sur le lien : <https://github.com/h2database/h2database/releases/download/version-2.4.240/h2-2025-09-22.zip> Sur le site principal, vous pouvez retrouver les tutoriels nécessaires pour comprendre son utilisation. Un autre lien vous présentant un petit exemple est donné sur le lien : <https://devstory.net/11895/installer-la-base-de-donnees-h2-et-utiliser-h2-console>

La base de données doit être lancée manuellement. Cette solution permet de démarrer une base de données externe, ce qui se rapproche de la réalité. Vous pouvez également retrouver sur le site de H2 des exemples de code permettant d'utiliser JDBC avec la base de données H2 pour la contacter depuis votre application Java : https://www.h2database.com/html/tutorial.html#connecting_using_jdbc